# 🔍 KCF Target Tracking and Autonomous Grasping

## 1. Course Overview

The function implemented in this lesson can be simply understood as: **mouse box-select any object, robotic arm automatically tracks the target, approaches and autonomously descends to grasp**.

Unlike color recognition or AprilTag machine-code methods, KCF (Kernelized Correlation Filters) is a **general-purpose prior-free target tracking algorithm** — without needing to pre-enter specific color thresholds or paste black and white QR codes. Simply box-select an object of interest in the real-time image, and the system establishes an appearance feature model for continuous stable tracking.

> **Important**
>
> **Important mechanism differences**:
>
> - **Different robotic arm initial poses**: USB monocular camera is close-range desktop top-down pose; 3D depth camera is forward-looking wide-range gimbal pose;
> - **This case recommends using 4*4*cm matching blocks**

## 2. Preparation and Running

This case requires opening 3 terminals to start the system sequentially:

### 2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

```
jetson@yahboom:~$ ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-09-195196-yahboom-1117638
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [camera_launch-1]: process started with pid [1117670]
[INFO] [kinemarics_dofbot-2]: process started with pid [1117672]
[INFO] [Ackman_driver_A1P-3]: process started with pid [1117674]
[kinemarics_dofbot-2] -----------------
[kinemarics_dofbot-2] fk
[kinemarics_dofbot-2] 5
[kinemarics_dofbot-2] -----------------
[camera_launch-1] [INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-10-139043-yahboom-1117670
[camera_launch-1] [INFO] [launch]: Default logging verbosity is set to INFO
[camera_launch-1] [INFO] [usb_cam_node_exe-1]: process started with pid [1117735]
[camera_launch-1] [usb_cam_node_exe-1] libEGL warning: DRI2: failed to authenticate
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.852521184] [usb_cam]: camera_name value: test_camera
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.852749618] [usb_cam]: framerate: 120.000000
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.876670665] [usb_cam]: camera calibration URL: package://usb_cam/config/cam
era_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.879330849] [usb_cam]: [test_camera] does not match narrow_stereo in file /
home/jetson/yahboomcar_ros2_ws/software/library_ws/install/usb_cam/share/usb_cam/config/camera_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.953127689] [usb_cam]: Starting 'test_camera' (/dev/video0) at 640x480 via
mmap (mjpeg2rgb) at 120 FPS
[camera_launch-1] [usb_cam_node_exe-1] [swscaler @ 0xaaaad038ffe0] No accelerated colorspace conversion found from yuv422p to rgb24.
[camera_launch-1] [usb_cam_node_exe-1] This device supports the following formats:
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 720 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1920 x 1080 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1024 x 768 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 640 x 480 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 800 x 600 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 1024 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 320 x 240 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 720 (9 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1920 x 1080 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1024 x 768 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 640 x 480 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 800 x 600 (20 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 1024 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 320 x 240 (30 Hz)
[Ackman_driver_A1P-3] [INFO] [1788337931.035307899] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.748886319] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.749045211] [usb_cam]: Setting 'exposure_auto' to 3
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.760676716] [usb_cam]: Setting 'focus_auto' to 0
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.836160385] [usb_cam]: Timer triggering every 8 ms
[camera_launch-1] [usb_cam_node_exe-1] [mjpeg @ 0xaaaad0382960] No JPEG data found in image
```

## 2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb grasp_usb
{'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610358, 'z_offset': 0.0}
----------------------------
x_offset:  0.009368814795573138
y_offset:  0.007718452024610358
z_offset:  0.0
----------------------
[INFO] [1788337928.183721749] [Grasp_Node]: Service not available, waiting again...
[INFO] [1788337929.186574716] [Grasp_Node]: Service not available, waiting again...
init done
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.489092034631593e-06, 1.3962632447969856, -2.82
89145258287586e-05]
get joint5 =  -90
callback:  dofbot_pro_interface.msg.AprilTagInfo(id=0, x=0.05222530663013458, y=0.009054901078343391, z=0.2150000035762787)
pose_T:  [ 0.1519739  -0.05698393  0.0175345 ]
pose_T_add_offset:  [ 0.16134271 -0.04926548  0.0175345 ]
pose_R:  (-2.9670595715936052, -7.795233443795971e-07, -1.5708201950476375)
Take it now.
```

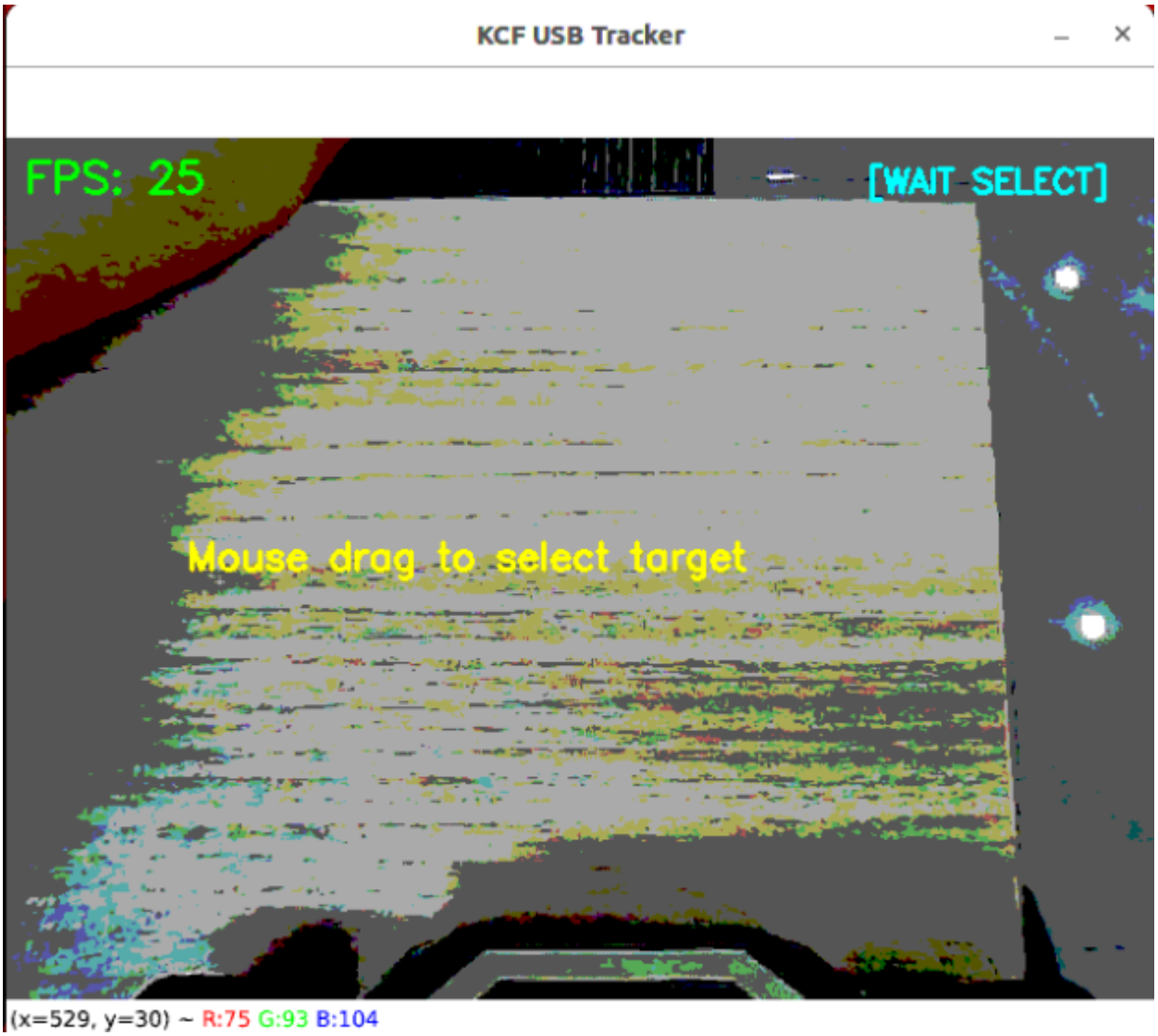
## 2.3 Start KCF Tracking Master Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver kcf_Tracker

# Or USB camera version
ros2 run dofbot_pro_usb kcf_follow
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb kcf_follow
--- KCF USB Tracking Node Initializing ---
[INFO] [1788343297.954450242] [kcf_usb_follow_node]: Initializing KCF USB Tracking and Grasping nod
e...
[INFO] [1788343297.957391740] [kcf_usb_follow_node]: object_size=0.0300m, dynamic_object_size=True
[INFO] [1788343297.958017136] [kcf_usb_follow_node]: KCF depth mode=calibrated_area, reference=73.0
x73.0px at 0.2250m, object_size=0.0300m
Successfully loaded offset_config: {'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610
358, 'z_offset': 0.0}
[INFO] [1788343298.276393898] [kcf_usb_follow_node]: KCF USB Tracking Node started! Please select t
arget with mouse in camera view.
```




Box-select object to start tracking:




# 3. Operation Demo and Interaction Flow (Version Difference Key Notes)

Due to hardware characteristics and algorithm implementation differences, **USB camera version** and **3D depth camera version** have significant differences in operation and robotic arm actions. Please read based on your device:

## 3.1 Solution A: USB Camera Version Interaction Flow (`kcf_follow`)

1. **Robotic arm initial positioning**: After program startup, the robotic arm smoothly moves to a high-angle downward pose: `[90°, 100°, 0°, 0°, 90°, 90°]`, directly vertical top-down view of desktop;
2. **Mouse box-select target**: In the image window, **hold left mouse button and drag** to draw a rectangle selecting the object to grasp. After releasing mouse, KCF algorithm immediately locks the target and draws 🟢 green tracking box;
3. **Manually press spacebar to trigger grasping**:
    - Keep the image window active, press the **spacebar (`Space`)** on the keyboard;
    - Chassis receives command and starts forward/backward linear velocity PID control, automatically and smoothly approaching the target (approaching to approximately 190mm);

4. **Grasping and unloading**: After reaching grasping distance, chassis stops, program analyzes target contour to calculate gripper alignment angle, robotic arm descends to grasp;
5. **Reset and reselect**: After grasping, chassis briefly reverses to avoid collision. Press keyboard `R` to re-box-select a new object.

---

## 3.2 Solution B: 3D Depth Camera Version Interaction Flow (`kcf_Tracker`)

1. **Robotic arm forward-looking patrol pose**: After program startup, the robotic arm moves to forward-looking wide-range gimbal pose: `[90°, 150°, 12°, 20°, 90°, 0°]` (joint 2 raised high to 150°, elbow extended, wider field of view);
2. **Mouse box-select target**: In the image window **hold left mouse button and drag** to box-select the target. After releasing mouse, the system completes KCF feature modeling;
3. **Robotic arm gimbal active dynamic servo tracking**:
    - **Manually press spacebar**! If you slowly move the target object horizontally or up/down, the robotic arm's joint 1 base servo and joints 2/3/4 pitching will rotate automatically, always keeping the lens facing the target;
4. **Continuous stable alignment auto-approach grasping (no spacebar needed)**:
    - When the target remains within the center tolerance range for continuous 10 frames (`ALIGN_COUNT_THRESHOLD = 10`), buzzer sounds to confirm lock;
    - **Chassis automatically starts and drives toward the target without user button press** (approaching to approximately 240mm optimal working zone);
    - After chassis is in position, directly read depth point cloud true distance, robotic arm automatically converts gripper deflection angle and descends to execute grasping;
5. **Motion reset**: After grasping, the robotic arm automatically returns to `[90°, 150°, 12°, 20°, 90°, 0°]` patrol pose, and resets alignment counter and PID status, ready to box-select the next target at any time.

> **Tip**
>
> When moving the tracked object, please **move smoothly and slowly**, avoid occlusion or fast shaking beyond the KCF filter's update window. If tracking is lost, press keyboard `R` to re-box-select at any time.

---

# 4. Core Source Code Analysis

The system is completed by "KCF target tracking decision" and "IK execution" two nodes working in coordination:

| Camera Type | KCF Tracking and Alignment Master Node | Grasping Action Execution Node |
|---|---|---|
| **USB Monocular Camera** | `dofbot_pro_usb/kcf_follow.py` | `dofbot_pro_usb/grasp_usb.py` |
| **3D Depth Camera** | `dofbot_pro_driver/KCF_Tracker.py` | `dofbot_pro_driver/grasp_desktop.py` |

The main technical implementation and control strategy comparison:

| Comparison Dimension | USB Monocular Solution (`kcf_follow.py`) | 3D Depth Camera Solution (`KCF_Tracker.py`) |
|---|---|---|
| **Robotic Arm Initial Pose** | `[90, 100, 0, 0, 90, 90]` (desktop top-down) | `[90, 150, 12, 20, 90, 0]` (forward-looking gimbal) |
| **Gimbal Following** | No (robotic arm joint angles fixed, static tracking) | Yes (with adaptive gimbal PID, each joint actively follows tracking) |
| **Approach Grasping Trigger** | **Manually press spacebar (`Space`)** to trigger chassis approach | **Continuous 10 frames center auto-trigger**, no manual button needed |
| **Distance Measurement Principle** | Box bounding box pixel size combined with prior size calculated via pinhole model | Physical depth sensor directly reads target center true distance |
| **Target Approach Distance** | Chassis PID control approaches to approximately 190mm | Chassis PID control approaches to approximately 240mm |
| **Spatial Calibration Compensation** | Superimpose X, Y 2D plane calibration deviation | Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation |

## 4.1 USB Monocular Camera Core Source Code Analysis (`kcf_follow.py`)

### 1. Mouse Drag Box Selection and KCF Initialization

When mouse is released, extract the box-selected rectangle ROI and call OpenCV tracker for initialization:

```
# kcf_follow.py —— Mouse box selection and KCF initial feature modeling
elif event == cv2.EVENT_LBUTTONUP:
    x1, y1 = min(start_x, x), min(start_y, y)
    w, h = abs(x - start_x), abs(y - start_y)
    if w > 10 and h > 10:
        self.select_roi = (x1, y1, w, h)
        self.state = State.TRACKING

# Initialize KCF
self.tracker = Tracker(tracker_type='KCF')
self.tracker.initWorking(rgb_image, self.select_roi)
```

### 2. Spacebar Trigger Chassis PID Approach

After user presses spacebar, the program switches from `TRACKING` to `APPROACHING` state. Chassis advances based on pinhole model estimated horizontal physical distance to 190mm:

```
# kcf_follow.py -- Spacebar trigger and chassis approach
if abs(dist_mm - GRASP_DIST_MM) > GRASP_TOLERANCE:
    linear_x = self.linearx_pid.compute(dist_mm, GRASP_DIST_MM)
    angle_z = self.angle_pid.compute(self.track_cx, ALIGN_TARGET_X)
    self._pub_vel(linear_x, 0, -angle_z)
else:
    self._pub_vel(0, 0, 0)
    self._trigger_grasp(rgb_image, cam_coords, est_depth)
```

---

## 4.2 3D Depth Camera Core Source Code Analysis (`KCF_Tracker.py`)

### 1. Robotic Arm Gimbal Adaptive Dynamic Closed-Loop Tracking (`XY_track`)

Unlike USB monocular solution, after box selection is complete in the depth camera solution, the robotic arm base and pitching joints perform active gimbal servo compensation following the target:

```
# KCF_Tracker.py -- Gimbal adaptive dynamic tracking
def XY_track(self, center_x, center_y):
    # Calculate deviation between target center and image center (320, 240)
    raw_angular_x = self.angular_x_PID.compute(center_x, TARGET_X)
    raw_angular_y = self.angular_y_PID.compute(center_y, TARGET_Y)

    # Slope limiter prevents severe overshoot and jitter
    diff_x = raw_angular_x - self.last_angular_x
    angular_x = self.last_angular_x + max(-self.max_step_change, min(self.max_step_change,
diff_x))

    # Dynamically calculate joint 1 base and joints 2/3/4 angles, lens always follows
    target_j1 = 90 + (10 * angular_x * -90.0)
    target_j2 = 150 + (angular_y * -150.0)
    target_j3 = 12 + (angular_y * -60.0)
    target_j4 = 20 + (angular_y * -100.0)
```

### 2. Continuous 10 Frames Lock Auto-Trigger Chassis Approach (Button-Free Mechanism)

The program detects whether the target enters the center tolerance range (`PIXEL_TOLERANCE = 10`). Once continuously stable for 10 frames, the program automatically switches to approach phase without user spacebar operation:

```
# KCF_Tracker.py -- 10 frames continuous center detection and auto-advance
err_x = abs(center_x - TARGET_X)
err_y = abs(center_y - TARGET_Y)
is_aligned = (err_x < PIXEL_TOLERANCE) and (err_y < PIXEL_TOLERANCE)

# Continuously meet alignment threshold, automatically enter approach
if is_aligned and self.Done_flag and not self.adjust_dist and not self.start_grasp:
    self.aligned_count += 1
    if self.aligned_count >= ALIGN_COUNT_THRESHOLD:  # Reach 10 frames
        self.Beep_Loop()        # Buzzer response confirmation
```

```
            self.adjust_dist = True # Automatically enter chassis approach phase
            self.XY_Track_flag = False

    if self.adjust_dist:
        # Drive chassis smoothly toward target based on depth map extracted true distance
    (240mm)
        if self.cur_distance > GRASP_DIST + 0.2:
            self.move_dist(self.cur_distance)
        else:
            self.adjust_dist = False
            self.start_grasp = True
            self.pubVel(0, 0, 0)
```

### 3. Hardware-Level Depth Point Cloud Reading and 3D IK Grasping

After chassis stops, directly extract the physical ranging data at the target center pixel `grasp_depth = depth_image[gy, gx] / 1000.0`, and calculate gripper alignment angle `compute_joint5(vx, vy)` through vertex slope vector. Publish coordinates to `grasp_desktop` node to execute descent closure and material grasping.

Through the above comparison analysis, users can clearly understand the deep-level differences in robotic arm pose and interaction control logic between the two hardware solutions, ensuring standardized operation under different hardware configurations.

## 4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

### 1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

### 2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
    - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
    - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
    - Gripper deviates right **increase value** (correct left)
    - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:

- Gripper too high in air and can't grasp **decrease value** (probe deeper)
- Gripper hits or scrapes table **increase value** (raise higher)